CirculAR — Scaffold técnico NestJS v1

1. Objetivo

Definir el scaffold técnico inicial del backend de CirculAR sobre NestJS para que la implementación arranque con una base consistente, modular, trazable y alineada con:

- blueprint inicial
- dominio oficial
- estados e invariantes
- contratos API v1
- arquitectura backend v1
- modelo relacional inicial
- catálogo de enums y constantes oficiales

Este documento fija:

- estructura real del proyecto
- módulos iniciales
- archivos base
- convenciones de naming
- setup de config
- setup de DB y migraciones
- setup de errores
- setup de auth
- setup de observabilidad
- setup de Swagger
- base para queues / outbox / realtime

---

2. Principios del scaffold

2.1 El scaffold no es decorativo

No es una carpeta linda. Debe bajar decisiones reales de arquitectura a una base técnica concreta.

2.2 Primero consistencia, después features

Antes de implementar listings, interactions o matches, tiene que existir una base transversal firme para:

- config

- errores
- auth
- persistencia
- módulos
- testing
- logging

2.3 El scaffold debe evitar drift futuro

Todo lo que se genere ahora debe ayudar a impedir:

- módulos mal cortados
- DTOs mezclados con entidades ORM
- lógica de dominio en controllers
- strings mágicos
- respuestas inconsistentes
- falta de ownership técnico

---

3. Stack técnico cerrado

- **Node.js LTS**
- **TypeScript**
- **NestJS**
- **PostgreSQL**
- **TypeORM**
- **Swagger / OpenAPI**
- **JWT + Argon2**
- **Redis + BullMQ**
- **WebSockets NestJS**
- **class-validator / class-transformer**
- **Jest o Vitest + Supertest**

---

4. Estructura de proyecto recomendada

```
circular-back/
  package.json
  tsconfig.json
  tsconfig.build.json
  nest-cli.json
  .env.example
  .eslintrc.js
  .prettierrc
  README.md
```

```
src/
  main.ts
  app.module.ts

  config/
    env/
      env.validation.ts
    app.config.ts
    database.config.ts
    auth.config.ts
    storage.config.ts
    queue.config.ts
    swagger.config.ts

  common/
    domain/
      enums/
      constants/
      types/
    application/
      base/
      ports/
    infrastructure/
      db/
        migrations/
        typeorm/
      events/
      queue/
      logging/
    presentation/
      decorators/
      dto/
      filters/
      guards/
      interceptors/
      pipes/

  modules/
    auth/
      application/
        commands/
        queries/
        services/
      domain/
        entities/
        value-objects/
        services/
```

```
  infrastructure/
    entities/
    repositories/
    strategies/
    mappers/
  presentation/
    controllers/
    dto/
    responses/
    auth.module.ts

profiles/
  application/
  domain/
  infrastructure/
  presentation/

listings/
  application/
  domain/
  infrastructure/
  presentation/
  read-models/

discovery/
  application/
  infrastructure/
  presentation/
  read-models/

interactions/
  application/
  domain/
  infrastructure/
  presentation/
  read-models/

matches/
  application/
  domain/
  infrastructure/
  presentation/
  read-models/

moderation/
  application/
  domain/
  infrastructure/
```

```
      presentation/

    reputation/
      application/
      domain/
      infrastructure/
      presentation/

    notifications/
      application/
      domain/
      infrastructure/
      presentation/

    health/
      presentation/
      health.module.ts

  shared/
    errors/
    kernel/
    result/
    utils/

test/
  e2e/
  fixtures/
  helpers/
```

---

5. Módulos iniciales reales para arrancar

No conviene crear todos los módulos con implementación completa desde el día 1. Sí conviene crear el esqueleto oficial de los más importantes.

5.1 Módulos a crear desde el arranque

- `AuthModule`
- `ProfilesModule`
- `ListingsModule`
- `ModerationModule`
- `HealthModule`
- `CommonModule` implícito vía carpetas compartidas
- `ConfigModule` global
- `DatabaseModule` o configuración DB centralizada

5.2 Módulos que pueden existir como placeholders al principio

- `DiscoveryModule`
- `InteractionsModule`
- `MatchesModule`
- `NotificationsModule`
- `ReputationModule`

Esto te permite mantener el corte arquitectónico correcto sin sobreimplementar demasiado temprano.

---

6. package.json base recomendado

Dependencias núcleo sugeridas:

```
{
  "dependencies": {
    "@nestjs/common": "^11.0.0",
    "@nestjs/config": "^4.0.0",
    "@nestjs/core": "^11.0.0",
    "@nestjs/jwt": "^11.0.0",
    "@nestjs/passport": "^11.0.0",
    "@nestjs/platform-express": "^11.0.0",
    "@nestjs/swagger": "^11.0.0",
    "@nestjs/typeorm": "^11.0.0",
    "@nestjs/websockets": "^11.0.0",
    "argon2": "^0.41.0",
    "bullmq": "^5.0.0",
    "class-transformer": "^0.5.1",
    "class-validator": "^0.14.1",
    "ioredis": "^5.0.0",
    "passport": "^0.7.0",
    "passport-jwt": "^4.0.1",
    "pg": "^8.12.0",
    "reflect-metadata": "^0.2.2",
    "rxjs": "^7.8.1",
    "typeorm": "^0.3.20"
  },
  "devDependencies": {
    "@nestjs/cli": "^11.0.0",
    "@nestjs/schematics": "^11.0.0",
    "@nestjs/testing": "^11.0.0",
    "@types/express": "^5.0.0",
    "@types/jest": "^29.5.12",
    "@types/node": "^22.0.0",
```

```
    "eslint": "^9.0.0",
    "jest": "^29.7.0",
    "prettier": "^3.3.0",
    "supertest": "^7.0.0",
    "ts-jest": "^29.2.5",
    "ts-loader": "^9.5.1",
    "ts-node": "^10.9.2",
    "tsconfig-paths": "^4.2.0",
    "typescript": "^5.6.0"
  }
}
```

---

7. Scripts recomendados

```
{
  "scripts": {
    "start": "nest start",
    "start:dev": "nest start --watch",
    "start:debug": "nest start --debug --watch",
    "build": "nest build",
    "lint": "eslint \"{src,test}/**/*.ts\" --fix",
    "test": "jest",
    "test:watch": "jest --watch",
    "test:e2e": "jest --config ./test/jest-e2e.json",
    "typeorm": "ts-node -r tsconfig-paths/register ./node_modules/typeorm/cli.js",
    "migration:generate": "npm run typeorm -- migration:generate
src/common/infrastructure/db/migrations/AutoMigration -d src/config/database.config.ts",
    "migration:create": "npm run typeorm -- migration:create
src/common/infrastructure/db/migrations/ManualMigration",
    "migration:run": "npm run typeorm -- migration:run -d src/config/database.config.ts",
    "migration:revert": "npm run typeorm -- migration:revert -d src/config/database.config.ts"
  }
}
```

---

8. Configuración de entorno

8.1 .env.example

```
NODE_ENV=development
PORT=3000
APP_NAME=CirculAR API
APP_VERSION=1.0.0
APP_COMMIT_SHA=local
```

```
DATABASE_HOST=localhost
DATABASE_PORT=5432
DATABASE_NAME=circular
DATABASE_USER=postgres
DATABASE_PASSWORD=postgres
DATABASE_SSL=false

JWT_ACCESS_SECRET=replace_me
JWT_ACCESS_EXPIRES_IN=15m
JWT_REFRESH_SECRET=replace_me
JWT_REFRESH_EXPIRES_IN=30d

REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_PASSWORD=

STORAGE_ENDPOINT=
STORAGE_REGION=
STORAGE_BUCKET=
STORAGE_ACCESS_KEY=
STORAGE_SECRET_KEY=
STORAGE_PUBLIC_BASE_URL=

SWAGGER_ENABLED=true
CORS_ORIGIN=http://localhost:3001
```

8.2 Validación de env

Crear `src/config/env/env.validation.ts` con Zod o class-validator.

Mi recomendación: **Zod**.

Debe validar al boot:

- presencia de variables críticas
- tipos correctos
- defaults razonables de desarrollo

---

9. app.module.ts

Debe ser muy simple y solo componer módulos.

```
@Module({
```

```
  imports: [
    ConfigModule.forRoot({
      isGlobal: true,
      validate: validateEnv,
      load: [appConfig, authConfig, databaseConfig, queueConfig, storageConfig],
    }),
    TypeOrmModule.forRootAsync({
      useFactory: buildTypeOrmOptions,
    }),
    AuthModule,
    ProfilesModule,
    ListingsModule,
    ModerationModule,
    HealthModule,
  ],
})
export class AppModule {}
```

Regla

`AppModule` no debe tener lógica de negocio ni providers “misc” desperdigados.

---

10. main.ts

Debe resolver la base transversal de la app.

Responsabilidades:

- bootstrap Nest app
- prefijo global `/v1`
- ValidationPipe global
- filtros globales de error
- interceptor de envelope success
- CORS
- Swagger
- shutdown hooks

Config recomendada

```
async function bootstrap() {
  const app = await NestFactory.create(AppModule, {
    bufferLogs: true,
  });

  app.setGlobalPrefix('v1');
```

```
  app.useGlobalPipes(
    new ValidationPipe({
      whitelist: true,
      forbidNonWhitelisted: true,
      transform: true,
      transformOptions: { enableImplicitConversion: true },
    }),
  );

  app.useGlobalFilters(new GlobalExceptionFilter());
  app.useGlobalInterceptors(new SuccessResponseInterceptor());

  app.enableCors({
    origin: process.env.CORS_ORIGIN?.split(',') ?? true,
    credentials: true,
  });

  setupSwagger(app);

  app.enableShutdownHooks();
  await app.listen(process.env.PORT ?? 3000);
}
```

---

11. Envelope estándar de respuesta

Esto tiene que salir desde el scaffold, no después.

11.1 Success

```
export interface SuccessEnvelope<T> {
  success: true;
  data: T;
  meta: Record<string, unknown>;
}
```

11.2 Error

```
export interface ErrorEnvelope {
  success: false;
  error: {
    code: string;
    message: string;
    details?: Record<string, unknown>;
```

```
  };
  meta: Record<string, unknown>;
}
```

11.3 Implementación

- `SuccessResponseInterceptor` envuelve responses normales
- `GlobalExceptionFilter` transforma errores a envelope homogéneo

---

12. Sistema de errores base

12.1 Clasificación recomendada

- `DomainError`
- `ApplicationError`
- `InfrastructureError`
- `ValidationAppError`
- `AuthorizationAppError`
- `NotFoundAppError`
- `ConflictAppError`

12.2 Archivo base sugerido

```
src/shared/errors/
  app-error.ts
  domain-error.ts
  conflict.error.ts
  not-found.error.ts
  forbidden.error.ts
  validation.error.ts
```

12.3 AppError base

```
export class AppError extends Error {
  constructor(
    public readonly code: string,
    public readonly message: string,
    public readonly statusCode: number,
    public readonly details?: Record<string, unknown>,
  ) {
```

```
    super(message);
  }
}
```

12.4 Regla clave

Los módulos no deben tirar `new Error('algo')` para reglas de negocio. Deben usar errores tipados alineados con el catálogo oficial.

---

13. Logging y tracing base

13.1 Objetivo mínimo

Cada request importante debería poder seguirse.

13.2 Qué incluir

- requestId
- method
- path
- statusCode
- durationMs
- userId si aplica
- errorCode si falla

13.3 Implementación sugerida

- middleware/interceptor de request context
- logger estructurado simple al inicio
- no hace falta meter OpenTelemetry de arranque

---

14. Database setup

14.1 database.config.ts

Debe exportar una `DataSource` usable por TypeORM CLI y por Nest.

14.2 Reglas de migraciones

- nunca usar `synchronize: true` fuera de tests efímeros
- siempre migraciones explícitas
- nombres claros y cronológicos
- una migración por cambio real de schema

14.3 Convención de nombres SQL

- tablas: `snake_case`
- columnas: `snake_case`
- FKs: `<table>_<column>_fk`
- uniques: `<table>_<field>_uq`
- índices: `idx_<table>_<field>`
- checks: `<table>_<rule>_chk`

---

15. Entidades ORM y dominio

15.1 Regla más importante

No usar entidades TypeORM como entidades de dominio.

Separar:

- ORM entity
- domain entity o aggregate state model
- read DTO

15.2 Dónde vive cada cosa

ORM entity

modules/listings/infrastructure/entities/listing.orm-entity.ts

Domain aggregate

modules/listings/domain/entities/listing.entity.ts

Mapper

modules/listings/infrastructure/mappers/listing.mapper.ts

---

16. Auth scaffold base

Este sí conviene dejarlo ya implementable de verdad.

16.1 Responsabilidades iniciales

- register
- login
- refresh
- logout
- me
- access token
- refresh token persistido en `sessions`

16.2 Archivos sugeridos

```
modules/auth/
  application/
    commands/
      register.command.ts
      login.command.ts
      refresh-session.command.ts
      logout.command.ts
    services/
      auth-command.service.ts
      auth-query.service.ts
  domain/
    entities/
      session.entity.ts
      auth-user.entity.ts
  infrastructure/
    entities/
      session.orm-entity.ts
      user.orm-entity.ts
    repositories/
      session.repository.ts
      user-auth.repository.ts
    strategies/
      jwt.strategy.ts
    services/
      token.service.ts
      password-hasher.service.ts
  presentation/
    controllers/
```

```
  auth.controller.ts
dto/
  register.dto.ts
  login.dto.ts
  refresh.dto.ts
responses/
  auth.response.ts
auth.module.ts
```

16.3 Endpoints mínimos

- `POST /v1/auth/register`
- `POST /v1/auth/login`
- `POST /v1/auth/refresh`
- `POST /v1/auth/logout`
- `GET /v1/auth/me`

Alineado directamente con Contratos API v1.

---

17. Profiles scaffold base

17.1 Primer alcance

- `GET /v1/profile/me`
- `PATCH /v1/profile/me`
- `PUT /v1/profile/me/reach-zones`
- `GET /v1/users/:userId/public-profile`

17.2 Modelado técnico inicial

Puede arrancar con:

- `public_profiles`
- `trust_profiles`
- `reach_zones`
- `reputation_profiles`

Pero dejando claro que:

- `reputation_profiles` es leído por profiles, no owned completamente por profiles
- recomputación la gobernará `reputation`

---

18. Listings scaffold base

18.1 Primer alcance real de código

- crear draft
- subir fotos
- editar listing editable
- submit review
- get detail
- get my listings
- pause
- resume
- archive

18.2 División interna sugerida

application

- `create-listing-draft.service.ts`
- `update-listing.service.ts`
- `submit-listing-review.service.ts`
- `pause-listing.service.ts`
- `resume-listing.service.ts`
- `archive-listing.service.ts`
- `renew-reservation.service.ts`
- `listing-query.service.ts`

infrastructure

- `listing.repository.ts`
- `garment.repository.ts`
- `listing-photo.repository.ts`
- `listing-read.repository.ts`

presentation

- `listings.controller.ts`
- DTOs request/response

---

19. Moderation scaffold base

No hace falta meter IA completa al inicio, pero sí conviene dejar listo el puerto.

19.1 Puerto clave

```
export interface ImageModerationProvider {
  auditImage(input: AuditImageInput): Promise<AuditImageResult>;
}
```

19.2 Implementación inicial

Primera versión puede ser:

- validaciones locales baratas
- adapter stub/mockeable
- worker preparado para reemplazarse luego por proveedor real

19.3 Jobs iniciales

- `audit-listing-photo.job`
- `resolve-listing-review.job`

---

20. Queue / Outbox scaffold base

20.1 Qué dejar listo desde el inicio

- conexión Redis
- queue module
- worker base
- outbox poller/processor base

20.2 No hace falta aún

- 20 colas diferentes
- arquitectura distribuida compleja

20.3 Sí hace falta

- una base clara para eventos operativos

- convención de payloads
- reintentos mínimos
- estado de procesamiento

---

21. WebSocket scaffold base

No hace falta desarrollar todo realtime todavía, pero sí dejar la puerta bien puesta.

21.1 Gateway inicial sugerido

- `notifications.gateway.ts`

21.2 Uso temprano

- notificaciones simples al usuario autenticado
- futuro: conversación contextual

21.3 Regla

WebSocket complementa. La verdad sigue estando en DB + API.

---

22. Swagger setup

22.1 Reglas

- un solo setup central
- título/version/contact claros
- bearer auth definido
- DTOs de request/response bien decorados

22.2 Objetivo

Que Swagger sea útil de verdad para frontend y para test manual, no un subproducto roto.

---

23. Health module

23.1 Endpoints sugeridos

- `GET /v1/health`
- `GET /v1/health/live`
- `GET /v1/health/ready`

23.2 Qué debería devolver `/health`

- `status`
- `timestamp`
- `service`
- `version`
- `commitSha`
- `environment`
- `database: up/down`

---

24. Testing scaffold

24.1 Unit

- domain services
- policies
- helpers de estado

24.2 Integration

- repositories
- transacciones críticas

24.3 E2E iniciales mínimos

- health responde
- register funciona
- login funciona
- `GET /auth/me` requiere auth

24.4 Regla

No dejar testing para “más adelante”. El scaffold ya tiene que nacer con base de test.

---

25. Convenciones de código

25.1 Controllers delgados

Los controllers:

- reciben DTO
- resuelven actor autenticado
- llaman application service
- devuelven response model

25.2 Application services orquestan

Ahí vive:

- ownership
- coordinación transaccional
- uso de repositorios
- eventos

25.3 Dominio no depende de Nest

Nada de decorators de Nest dentro del dominio.

25.4 Repositories por ownership

No dejar acceso anárquico a cualquier tabla desde cualquier módulo.

25.5 Read models separados

Las responses de UI deben salir de query services/read repositories, no de entidades ORM crudas.

---

26. Orden recomendado de implementación real después del scaffold

Fase 1

- bootstrap repo
- config
- DB
- migrations
- health
- error handling
- auth

Fase 2

- profiles base
- listings draft flow
- upload photos
- submit review

Fase 3

- moderation mínima
- listing publication flow
- listing detail
- my listings

Fase 4

- discovery
  n- saved listings
- dismiss

Fase 5

- purchase intents
- trade proposals
- incoming interactions
- aceptación / rechazo transaccional

Fase 6

- match session
- conversation thread
- messages
- quick actions
- confirm success / fail / cancel

Fase 7

- notifications
- reputation
- jobs automáticos
- reservation expiration
- archive técnico

---

27. Archivos mínimos concretos que deberían existir primero

src/main.ts
src/app.module.ts
src/config/env/env.validation.ts
src/config/app.config.ts
src/config/database.config.ts
src/config/auth.config.ts
src/config/swagger.config.ts

src/common/presentation/filters/global-exception.filter.ts
src/common/presentation/interceptors/success-response.interceptor.ts
src/common/presentation/guards/jwt-auth.guard.ts

src/shared/errors/app-error.ts
src/shared/errors/conflict.error.ts
src/shared/errors/not-found.error.ts
src/shared/errors/forbidden.error.ts
src/shared/errors/validation.error.ts

src/modules/health/presentation/health.controller.ts
src/modules/health/health.module.ts

src/modules/auth/presentation/controllers/auth.controller.ts
src/modules/auth/presentation/dto/register.dto.ts
src/modules/auth/presentation/dto/login.dto.ts
src/modules/auth/presentation/dto/refresh.dto.ts
src/modules/auth/application/services/auth-command.service.ts
src/modules/auth/application/services/auth-query.service.ts
src/modules/auth/infrastructure/services/token.service.ts
src/modules/auth/infrastructure/services/password-hasher.service.ts
src/modules/auth/infrastructure/strategies/jwt.strategy.ts
src/modules/auth/auth.module.ts

---

28. Primera versión de roadmap de código inmediato

Sprint técnico 0

Objetivo: repo levantado y sano.

Debe quedar:

- app levanta
- DB conecta
- migraciones corren
- health responde
- swagger abre
- register/login/me funcionando
- error envelope consistente

Sprint 1

Objetivo: profiles + listings base.

Debe quedar:

- profile me
- patch profile
- reach zones
- create listing draft
- upload photos
- submit review
- get listing detail
- get my listings

---

29. Decisiones cerradas de scaffold v1

Cerrado

- NestJS monolito modular
- TypeORM con migraciones
- configuración validada al boot
- error envelope estándar global
- success interceptor global
- auth JWT + refresh persistido
- Swagger desde el inicio
- health endpoints reales
- base de Redis/BullMQ y outbox
- separación entre domain / application / infrastructure / presentation
- DTOs y read models separados de entidades ORM

No hacer todavía

- generadores mágicos de código para todo
- microservicios
- CQRS ceremonial
- event sourcing
- upload presigned complejo
- realtime full chat desde el primer commit

---

30. Resultado esperado de esta etapa

Con este scaffold técnico NestJS v1, CirculAR queda listo para pasar de documentación a implementación real sin improvisación estructural.

La siguiente etapa ya es directamente:

1. generar el repo NestJS base
2. crear la estructura de carpetas real
3. implementar bootstrap técnico
4. levantar el primer set de endpoints reales

---

31. Próximo paso recomendado

Después de este documento, lo correcto ya no es otro documento conceptual.

Lo correcto es:

empezar a generar código real

Orden recomendado inmediato:

1. bootstrap técnico del proyecto NestJS
2. auth base
3. health + swagger + config + db
4. luego profiles y listings